LAPORAN PRAKTIKUM
TEKNOLOGI IOT II



DI SUSUN OLEH :

NAMA : DEA ZUL HAFNI SHERLYZA

NPM : 25782065

KELAS : TRI 3 C

PRODI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026

Modul 7: Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

7.5 Praktikum

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.

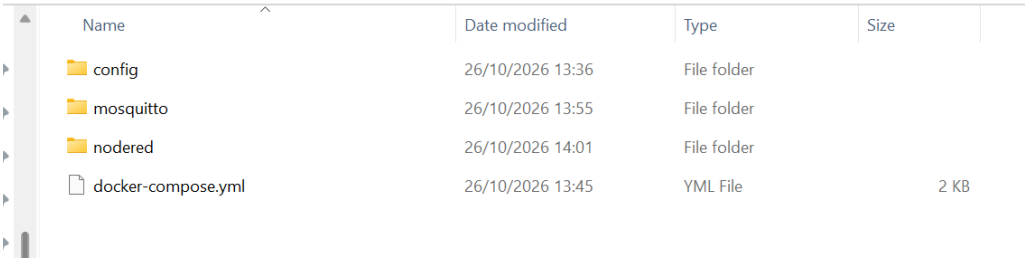
Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).
2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual:

   iot-server/

   ├── mosquitto/

   │ └── config/
3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama **mosquitto.conf**, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:




```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

4. Kembali ke folder induk iot-server, buat berkas baru bernama **docker-compose.yml**, lalu salin konfigurasi layanan terpadu berikut:

File Edit View

```
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:

cd D:\iot-server

6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):

docker compose up -d

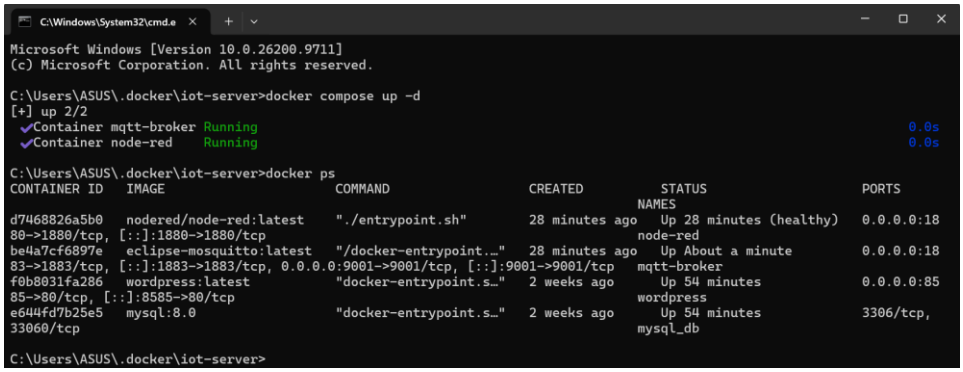
(Catatan: Pada versi Docker lama, gunakan perintah docker-compose up -d).

7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.

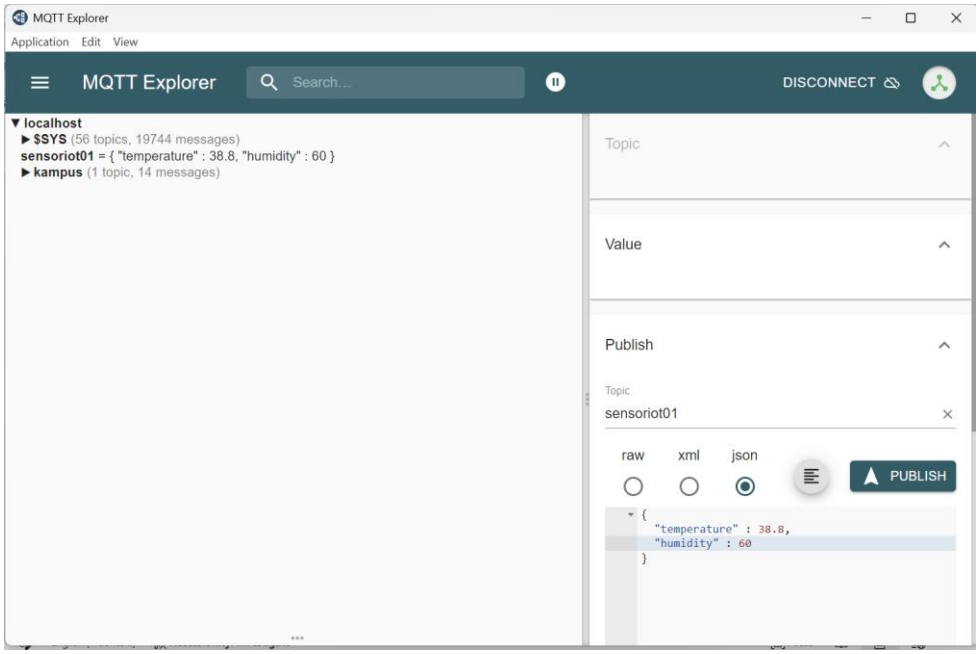
8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:

docker ps

Pastikan kolom STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).



9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: **http://localhost:1880**.
   Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!

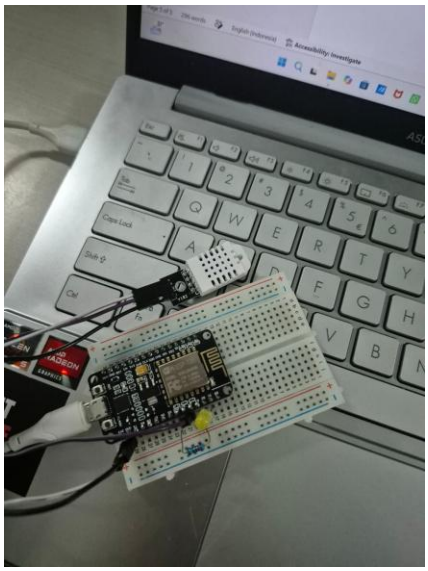


7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

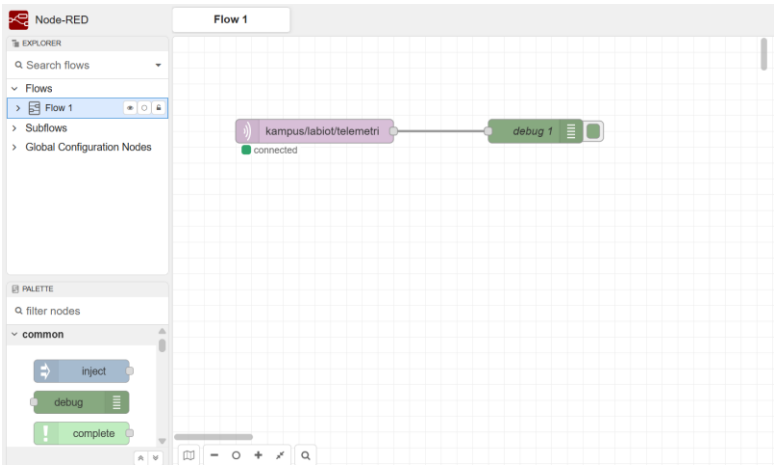


Dokumentasi :



7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

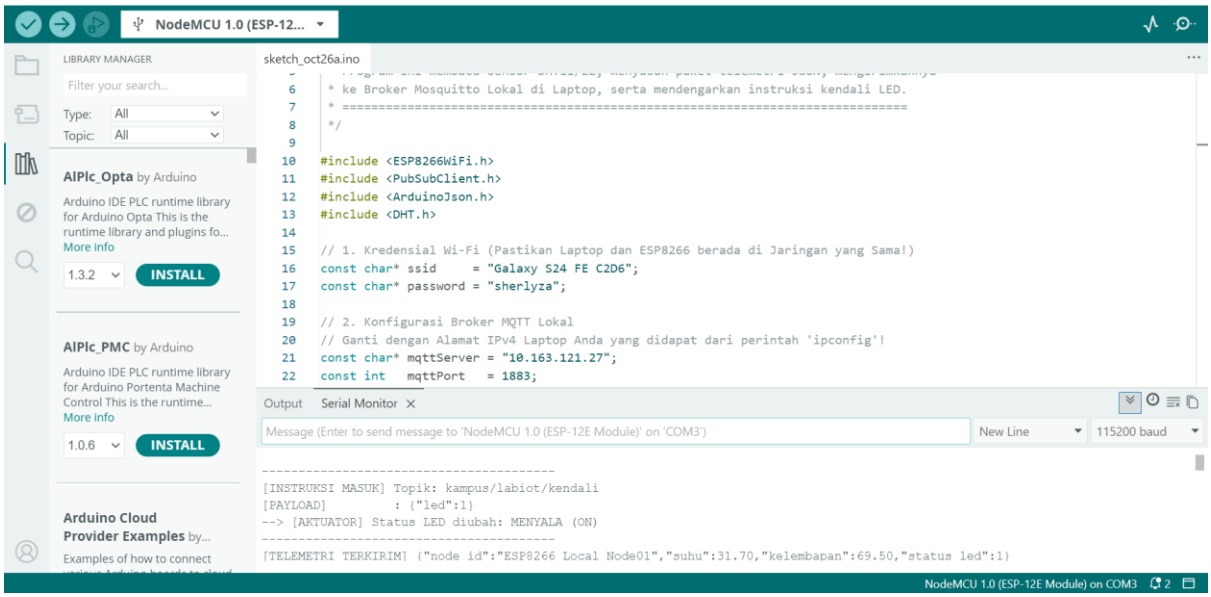
Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.



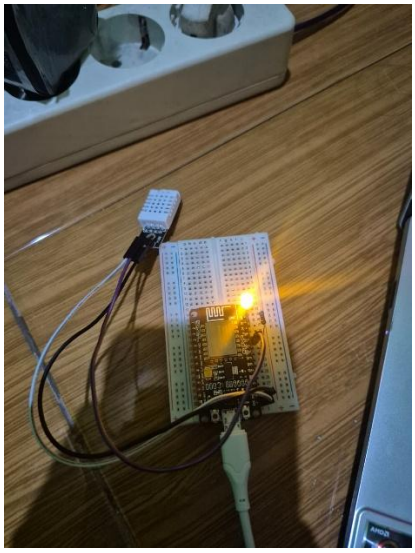
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

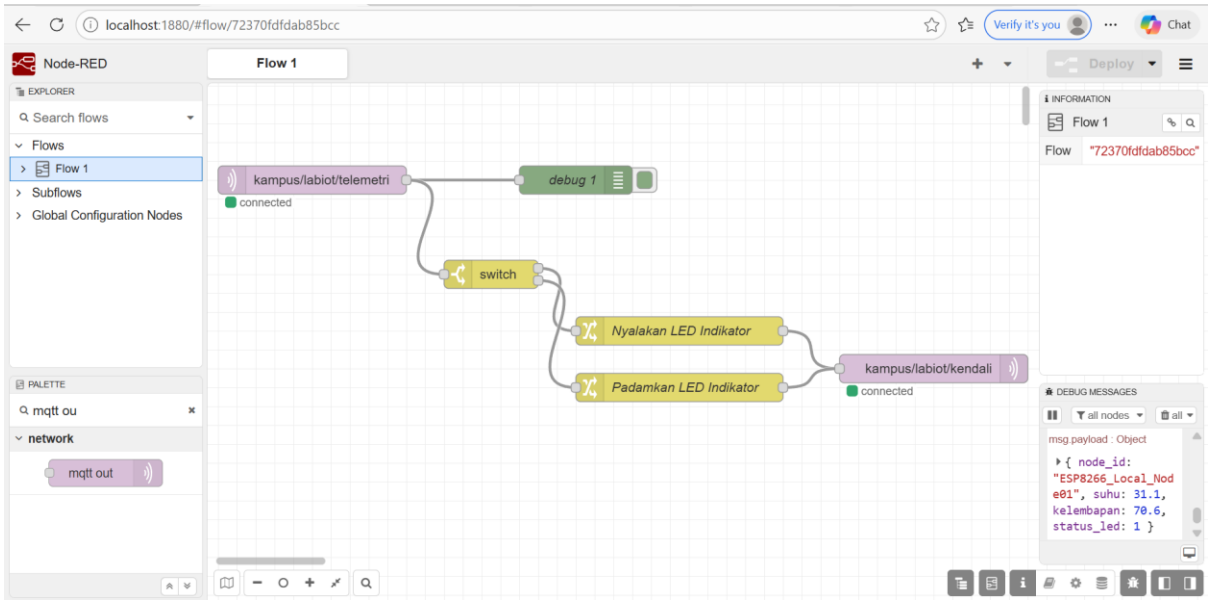
Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.



Dokumentasi :





7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- Latihan1 (Analisis Jaringan Internal Docker & Service Discovery): Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

    i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

    Karena di dalam ekosistem Docker, setiap kontainer punya network namespace sendiri yang terisolasi. Kalau kita nulis localhost, artinya Node-RED lagi nyari

layanan MQTT di dalam kontainer Node-RED itu sendiri, bukan di laptop fisik kita atau di kontainer sebelah. Padahal di dalam kontainer Node-RED ya jelas nggak ada broker Mosquitto yang jalan, makanya langsung ditolak (Connection Refused).

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

Jadi, waktu kita pakai Bridge Network bawaan Docker Compose, Docker otomatis nyiapin jaringan internal buat kontainer-kontainer yang kita buat. Di sinilah Embedded DNS Server si Docker beraksi. DNS internal ini otomatis ngenalin dan nge-resolve nama service (misalnya kita tulis mqtt atau mqtt-broker di settingan Node-RED) langsung ke alamat IP internal kontainer Mosquitto tersebut di dalam jaringan Docker yang sama. Makanya kita nggak perlu pusing nyari IP address kontainernya satu-satu, cukup panggil nama layanannya aja

- Latihan2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall): Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  Karena beda segmen jaringan / subnet! Laptop ada di subnet 192.168.1.x (misal dari Wi-Fi rumah/kampus), sementara ESP8266 dapet IP 192.168.43.x (biasanya dari hotspot HP). Karena beda network, paket data dari ESP8266 nggak bakal tahu harus dikirim ke mana di layer jaringan lokal tanpa adanya routing khusus, jadi ya bakal timeout atau nyasar.

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

  Kita harus nyambungin kedua perangkat ke Access Point / Wi-Fi yang sama.

  Pilihannya: Entah laptop yang dipindahin koneksinya ke hotspot HP yang sama dengan ESP8266, atau settingan Wi-Fi di program ESP8266 diganti supaya nyambung ke Wi-Fi rumah/kampus yang lagi dipakai laptop. Intinya, SSID dan password routernya harus klop biar dapet IP di satu rentang subnet yang sama.

- Latiha3 (Implementasi Filter Histeresis pada Rule Engine): Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}).
  Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
// Ambil status LED sebelumnya dari context, default-nya 0 (padam)
let ledStatus = context.get('ledStatus') || 0;
let suhu = parseFloat(msg.payload);

// Logika Histeresis
if (suhu > 31.5) {
    ledStatus = 1; // Nyala kalau suhu > 31.5°C
} else if (suhu < 30.0) {
    ledStatus = 0; // Padam kalau suhu turun sampai < 30.0°C
}
// Di antara 30.0 - 31.5, status ledStatus dibiarkan tetap (mempertahankan state sebelumnya)

// Simpan kembali statusnya ke context
context.set('ledStatus', ledStatus);

// Kirim output ke node berikutnya
msg.payload = {"led": ledStatus};
return msg;
```

- Latihan4 (Analisis Mekanisme Volume Mount & Persistensi Data): Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.
  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  Semua flow (kanvas visual) yang udah capek-capek kita bikin di Node-RED bakal hilang total / ke-reset! Kenapa? Karena kontainer Docker itu sifatnya

ephemeral (sementara). Kalau nggak dipasang volume mount, semua data yang disimpen di dalam kontainer bakal musnah pas kontainer dimatiin pake docker compose down

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

ind Mount: Kita secara manual nentuin folder spesifik yang ada di laptop fisik kita (host), contohnya kayak `./nodered:/data`. Jadi filenya bisa kita lihat dan edit langsung lewat File Explorer Windows.

Named Volume: Pengelolaannya diatur sepenuhnya sama Docker di dalam area penyimpanan internal Docker (biasanya di direktori `/var/lib/docker/volumes/...`). Kita nggak perlu pusing mikirin path foldernya di laptop, dan cara ini biasanya lebih aman dari risiko kehapus tidak sengaja sama user.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

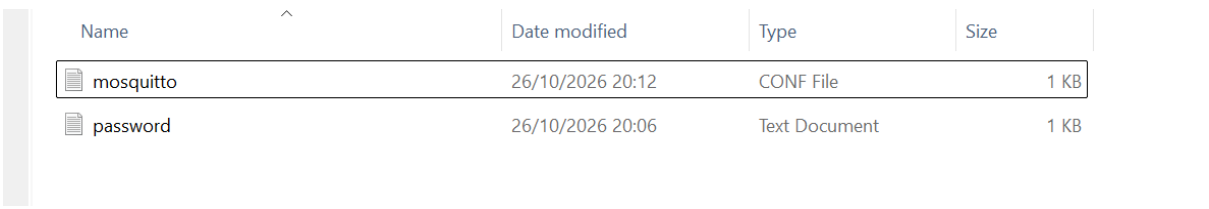
- **Tujuan:** Mampu mengonfigurasi berkas kata sandi terenkripsi di dalam kontainer Mosquitto, menonaktifkan akses anonim, menganalisis dampak penolakan koneksi (*rc=5*), serta memperbarui konfigurasi klien ESP8266 dan Node-RED secara mandiri.

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

```
C:\Users\ASUS\.docker\iot-server>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/
```



2. Memperbarui Konfigurasi Kebijakan Mosquitto:

File Edit View

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true
#password_file /mosquitto/config/password.txt

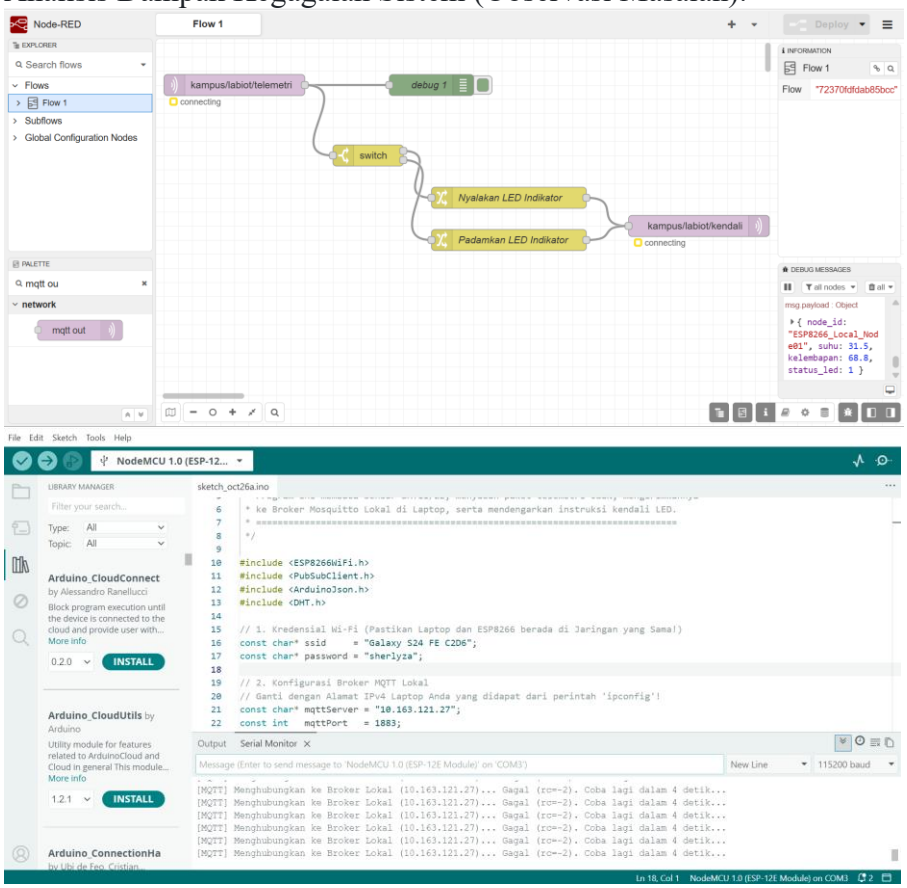
# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Memuat Ulang Layanan Broker:

```
C:\Users\ASUS\.docker\iot-server>docker restart mqtt-broker
mqtt-broker
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):



5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

